

CS3205: Assignment 4 Report

Distance Vector Routing (DVR) Protocol

Ishayu Ghosh - EE22B109

April 27, 2026

1 Introduction

This report details the implementation and analysis of a Distance Vector Routing (DVR) protocol in C++. The baseline code, which utilized uniform hop-count metrics, was extended to support weighted links, implement the Poisoned Reverse technique to mitigate routing loops, and simulate network reconvergence under a simulated link failure.

2 Task 1: Support Weighted Routing

The primary objective of this task was to transition the routing protocol from a simple hop-count mechanism to a weighted path-cost mechanism.

2.1 Implementation Details

- **Data Structure Extension:** The `NetInterface` class in `node.h` was modified to include a `linkCost` integer attribute, complete with appropriate getter and setter methods. The `addInterface` method signature was also updated to accept this cost during network initialization.
- **Input Parser Modification:** The network topology parser in `main.cpp` was extended to extract the integer link cost as the fourth argument of each connection declaration.
- **Routing Logic Update:** In `routing_algo.cpp`, the `recvMsg()` function was refactored. When a node receives a distance vector, it extracts the cost of the link over which the message arrived. It then evaluates prospective routes using the equation:

$$New_Cost = Received_Cost + Link_Cost$$

The routing table is strictly updated if `new_cost` is strictly less than the current cost for that destination.

3 Task 2: Implement Poisoned Reverse

Distance Vector algorithms are inherently susceptible to routing loops, specifically the “Count-to-Infinity” problem. To resolve this, the Poisoned Reverse heuristic was integrated.

3.1 Implementation Details

- **Defining Infinity:** A global constant `INF = 999` was defined to represent an unreachable node.
- **Neighbor-Specific Vectors:** The `sendMsg()` function in `node.h` was overhauled. Instead of broadcasting a uniform routing table to all interfaces, it now generates a customized table for each neighbor. If a node's route to destination D utilizes neighbor Y as its next hop, it advertises a cost of `INF` to neighbor Y . For all other neighbors, it advertises its true cost.
- **Forced Path Updates:** The `recvMsg()` logic was updated to bypass standard shortest-path checks if the incoming update is from the current next-hop router. If the next-hop router advertises a new cost (even an infinite one), the node must unconditionally adopt it to immediately break defunct paths.

4 Task 3: Simulate Link Failure and Observations

To validate the Poisoned Reverse implementation, a dynamic link failure was simulated after the network's initial convergence.

4.1 Simulation Methodology

A helper function `updateInterfaceCost` was added to the `Node` class. In `main.cpp`, after the first convergence, the bidirectional link between Node A (10.0.0.1) and Node B (10.0.0.21) was artificially severed by setting its interface cost to `INF`. The algorithm was then resumed to track the reconvergence process.

4.2 Observations

The network successfully reconverged in exactly 2 rounds.

Upon the failure of the $A \leftrightarrow B$ link, Node A became completely isolated, instantly updating its routing table entries to 999. Node B lost its direct connection to A. However, because Poisoned Reverse was active, Node C had been broadcasting an infinite cost to B for reaching A (since C's route to A went through B).

Consequently, when the direct link failed, Node B did not mistakenly attempt to route through C. It immediately recognized the path was dead and updated its cost to A to 999. The immediate propagation of infinite costs prevented any routing loops, validating the robust implementation of the modified protocol.

5 Conclusion

The implemented modifications successfully adapted the standard DVR algorithm to handle realistic network conditions. The introduction of link weights allows for traffic-aware routing, while Poisoned Reverse ensures immediate structural updates during link failures, effectively solving the count-to-infinity flaw.